Database Management – SQL & PL/SQL

- **Dhaval Jethva**

Theme: Personal Expense Tracker DB

Section A: Concept Application

1. **Explain how relational databases help maintain accuracy in expense records.**

   ANS: Relational databases help maintain accuracy in expense records by connecting related data through tables and keys. They reduce duplicate data, enforce valid relationships, and ensure that each expense is linked to the correct user and category. This helps keep data organized, consistent, and error-free.

2. **Why are constraints important in personal finance data?**

   ANS: Constraints are important in personal finance data because they help prevent errors and keep records accurate. For example, constraints can ensure that expense amounts are positive, user IDs are valid, and required fields are not left empty. This helps maintain reliable and consistent financial information.

3. **How does GROUP BY help analyze spending patterns?**

ANS: GROUP BY helps analyze spending patterns by organizing expenses into categories, dates, or users and then calculating totals for each group. For example, it can show how much money was spent on food, travel, or shopping. This makes it easier to track and compare spending habits.

Example:

```
SELECT category_id, SUM(amount)
FROM expenses
GROUP BY category_id;
```

This query shows the total expense for each category.

4. **Explain a scenario where rollback is required during expense entry.**

ANS : A **rollback** is required when an error occurs during expense entry and the transaction should not be saved partially.

Example:
Suppose a user enters an expense of ₹500 and the system also updates the monthly budget. If the expense is saved but the budget update fails due to an error, a **rollback** will undo the expense entry as well. This keeps the data accurate and consistent.

5. **How do views help users track monthly expenses efficiently?**

   ANS: Views help users track monthly expenses efficiently by showing frequently needed data in a single virtual table. They simplify complex queries and allow users to quickly see monthly spending summaries without writing SQL each time.

Example: A view can display the total expenses for each month, making it easier to monitor and analyze spending patterns.

6. **Why use triggers for automatic category or balance updates?**

ANS: Triggers are used to automatically perform actions when data is inserted, updated, or deleted in a database. They can automatically update account balances or expense categories, ensuring that records remain accurate and consistent. This reduces manual work, saves time, and helps prevent errors in the system.

Section B: SQL Hands-On

1. DDL Understanding

1. **Explain why foreign keys are used: Specifically, why must user_id in the expenses table correspond to a real ID in the users table?**

   ANS: Foreign keys are used to maintain data accuracy and relationships between tables. In the **expenses** table, the **user_id** must match a real **user_id** in the **users** table so that every expense belongs to a valid user. This prevents invalid or duplicate records and helps keep the database consistent and reliable.

2. **Mention one issue that would occur if foreign keys were removed: Describe the risk of "Orphaned Records" (e.g., an expense existing for a user who has been deleted).**

ANS: If foreign keys were removed, **orphaned records** could occur. For example, if a user is deleted from the **users** table, their expenses might still remain in the **expenses** table. These expense records would no longer be linked to any valid user, causing inaccurate data and making the database harder to manage.

2. DML Operations

- **Insert 5 users into the users table with unique names and emails.**

  INSERT INTO users VALUES

  (1,'Dhaval','dhaval@gmail.com','2026-01-01');

  INSERT INTO users VALUES

  (2,'Raj','raj@gmail.com','2026-01-02');

  INSERT INTO users VALUES

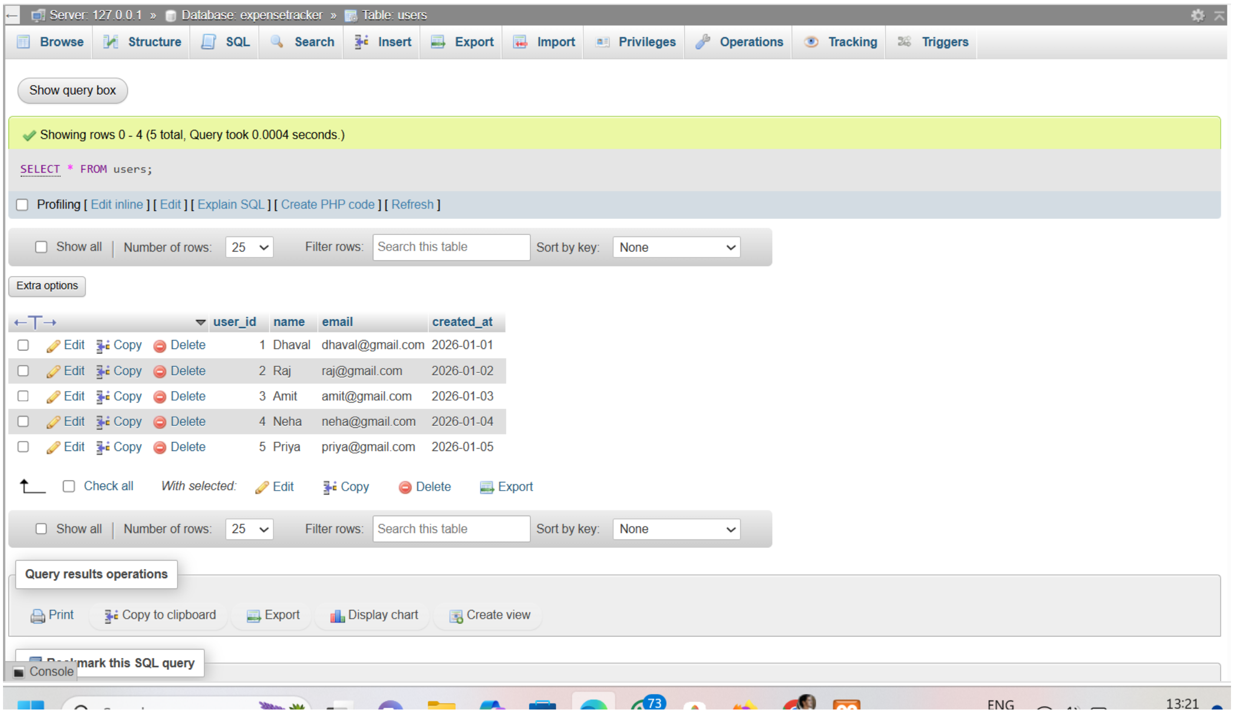
  (3,'Amit','amit@gmail.com','2026-01-03');

  INSERT INTO users VALUES

  (4,'Neha','neha@gmail.com','2026-01-04');

  INSERT INTO users VALUES

  (5,'Priya','priya@gmail.com','2026-01-05');

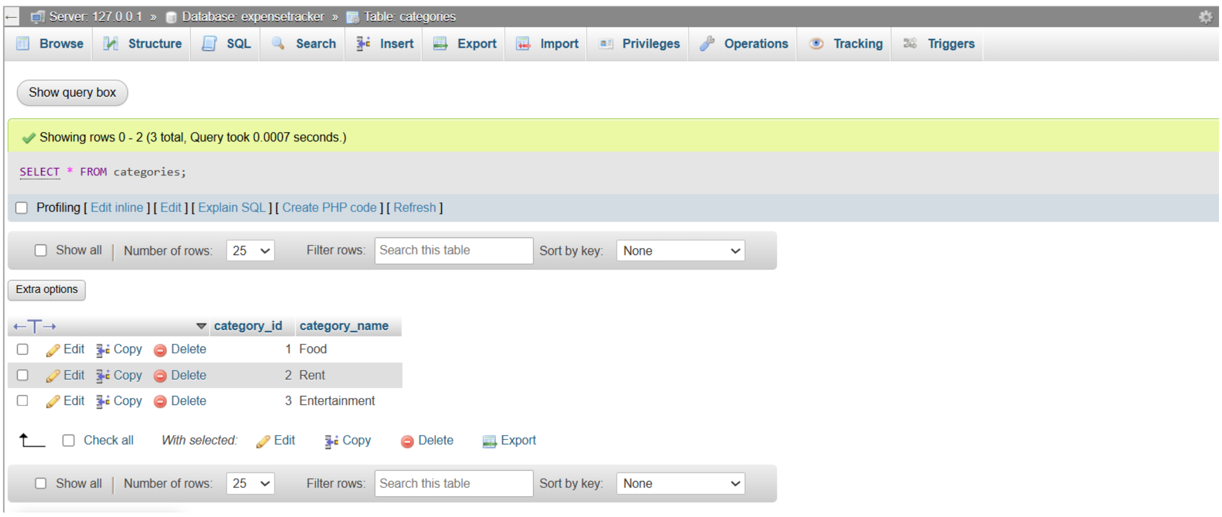


- **Insert 3 categories (e.g., 'Food', 'Rent', 'Entertainment').**

  INSERT INTO categories VALUES(1,'Food');

  INSERT INTO categories VALUES(2,'Rent');

  INSERT INTO categories VALUES(3,'Entertainment');



10 expense records distributed across different users and categories.

INSERT INTO expenses VALUES(101,1,1,500,'2026-05-01');

INSERT INTO expenses VALUES(102,1,2,8000,'2026-05-02');

INSERT INTO expenses VALUES(103,2,1,300,'2026-05-03');

INSERT INTO expenses VALUES(104,2,3,1200,'2026-05-04');

INSERT INTO expenses VALUES(105,3,1,700,'2026-05-05');

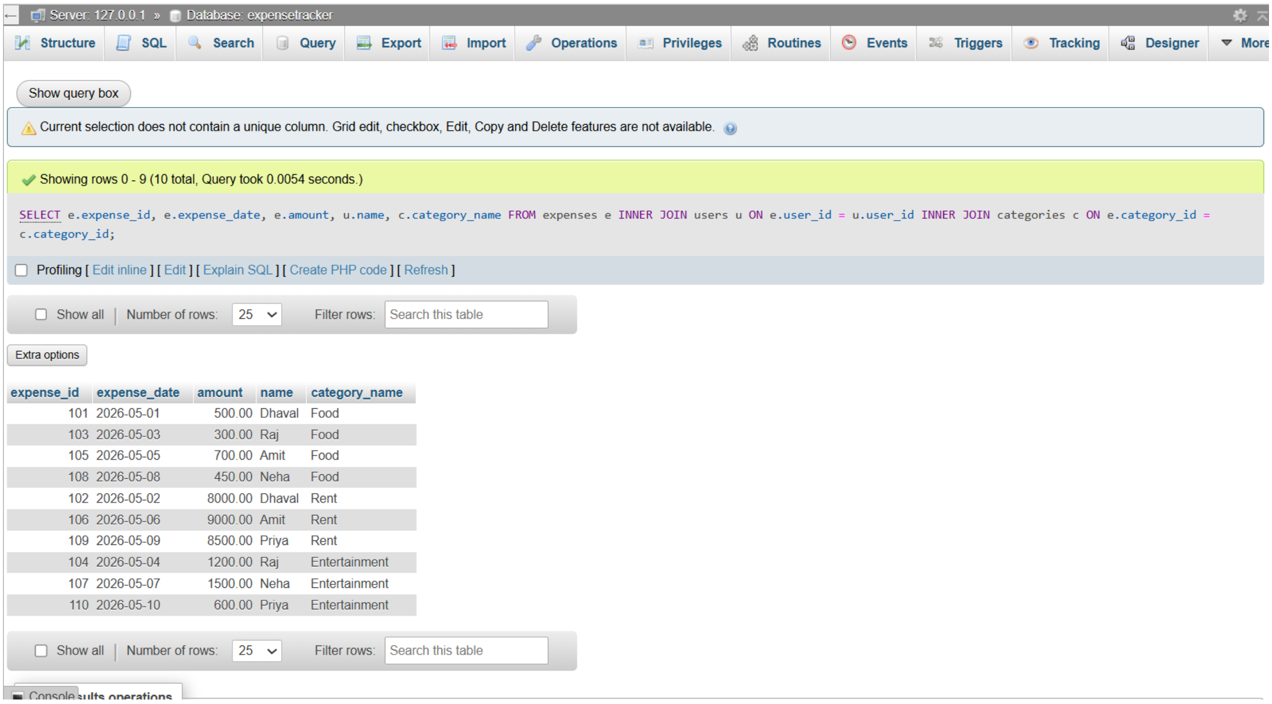
INSERT INTO expenses VALUES(106,3,2,9000,'2026-05-06');

INSERT INTO expenses VALUES(107,4,3,1500,'2026-05-07');

INSERT INTO expenses VALUES(108,4,1,450,'2026-05-08');

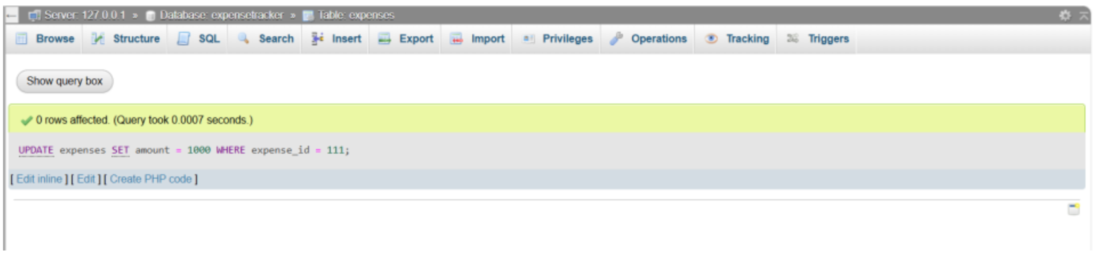
INSERT INTO expenses VALUES(109,5,2,8500,'2026-05-09');

INSERT INTO expenses VALUES(110,5,3,600,'2026-05-10');

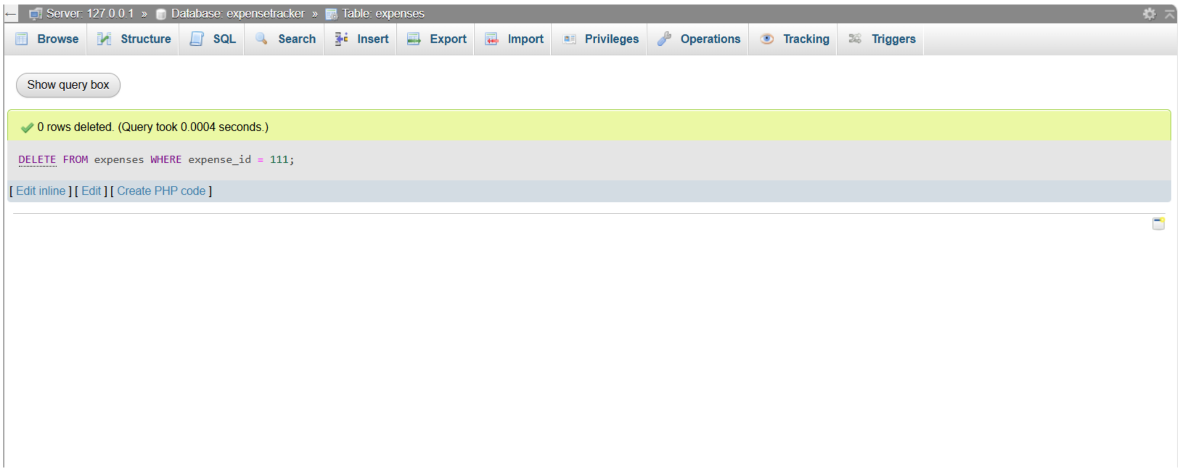


expense_id	expense_date	amount	name	category_name
101	2026-05-01	500.00	Dhaval	Food
103	2026-05-03	300.00	Raj	Food
105	2026-05-05	700.00	Amit	Food
108	2026-05-08	450.00	Neha	Food
102	2026-05-02	8000.00	Dhaval	Rent
106	2026-05-06	9000.00	Amit	Rent
109	2026-05-09	8500.00	Priya	Rent
104	2026-05-04	1200.00	Raj	Entertainment
107	2026-05-07	1500.00	Neha	Entertainment
110	2026-05-10	600.00	Priya	Entertainment

- **Update one incorrect expense: Use the UPDATE command to change the amount of a specific expense_id.**

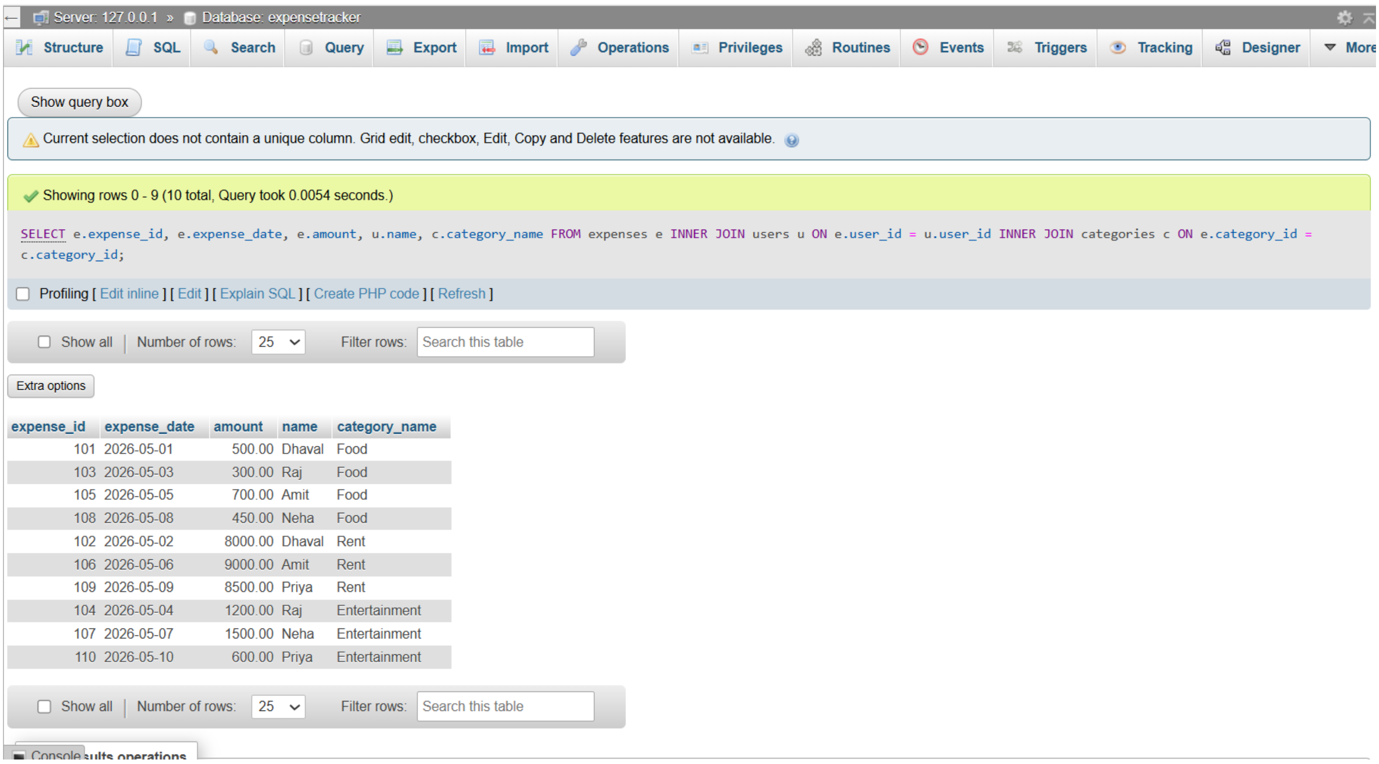


- **Delete one expense: Remove a record where the amount is less than a specific value.**



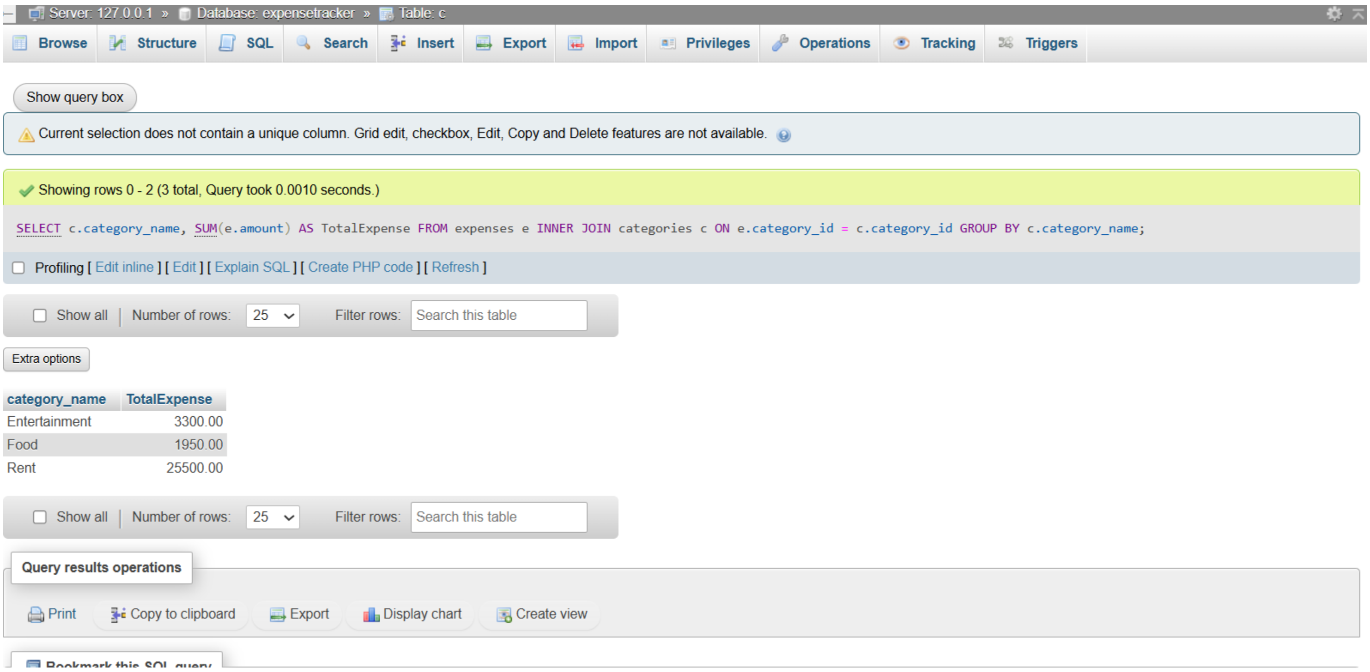
3. Data Retrieval

- **Display all expenses with details: Show the expense_date, amount, name (from users), and category_name (from categories) using INNER JOIN.**

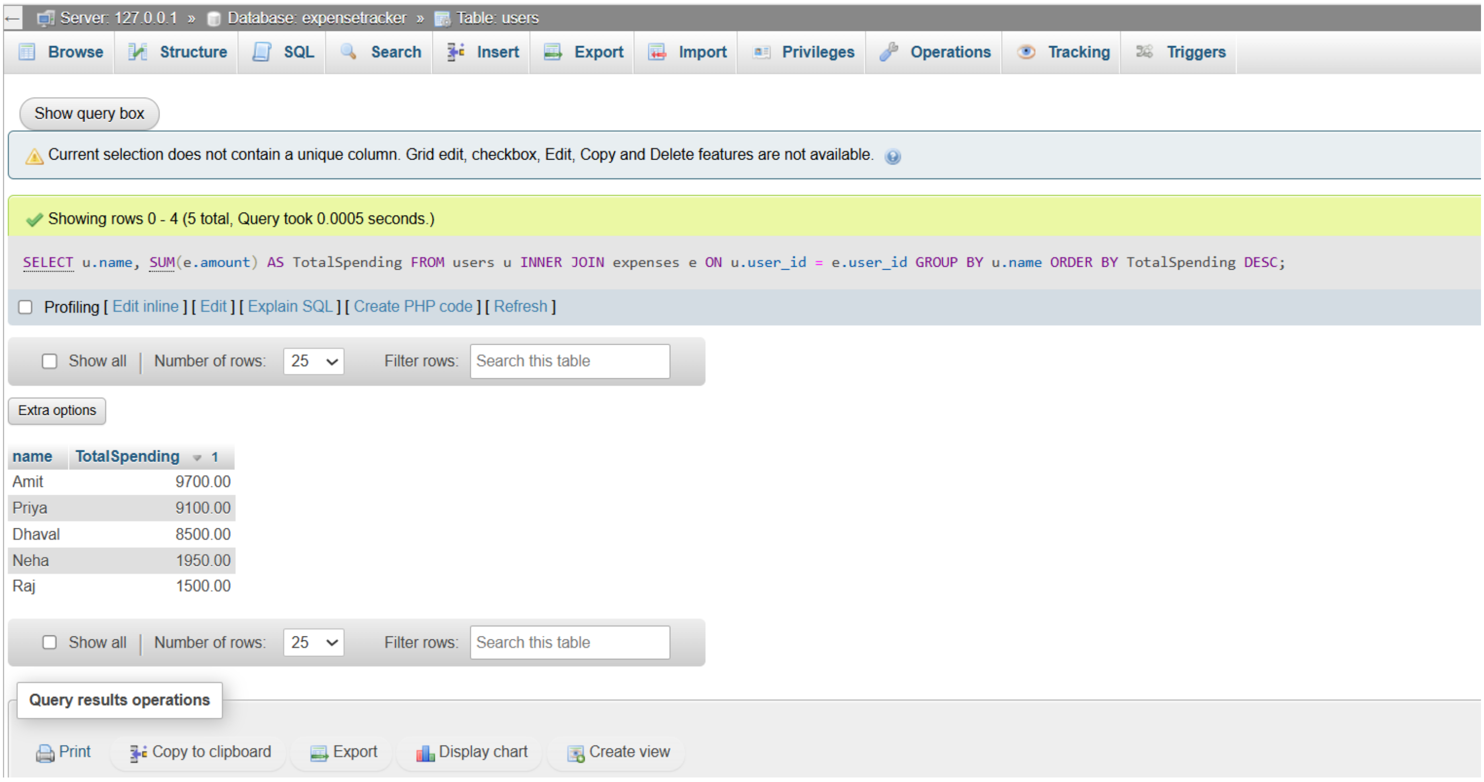


expense_id	expense_date	amount	name	category_name
101	2026-05-01	500.00	Dhaval	Food
103	2026-05-03	300.00	Raj	Food
105	2026-05-05	700.00	Amit	Food
108	2026-05-08	450.00	Neha	Food
102	2026-05-02	8000.00	Dhaval	Rent
106	2026-05-06	9000.00	Amit	Rent
109	2026-05-09	8500.00	Priya	Rent
104	2026-05-04	1200.00	Raj	Entertainment
107	2026-05-07	1500.00	Neha	Entertainment
110	2026-05-10	600.00	Priya	Entertainment

- **Show total expense amount per category: Use SUM(amount) and GROUP BY category_name.**



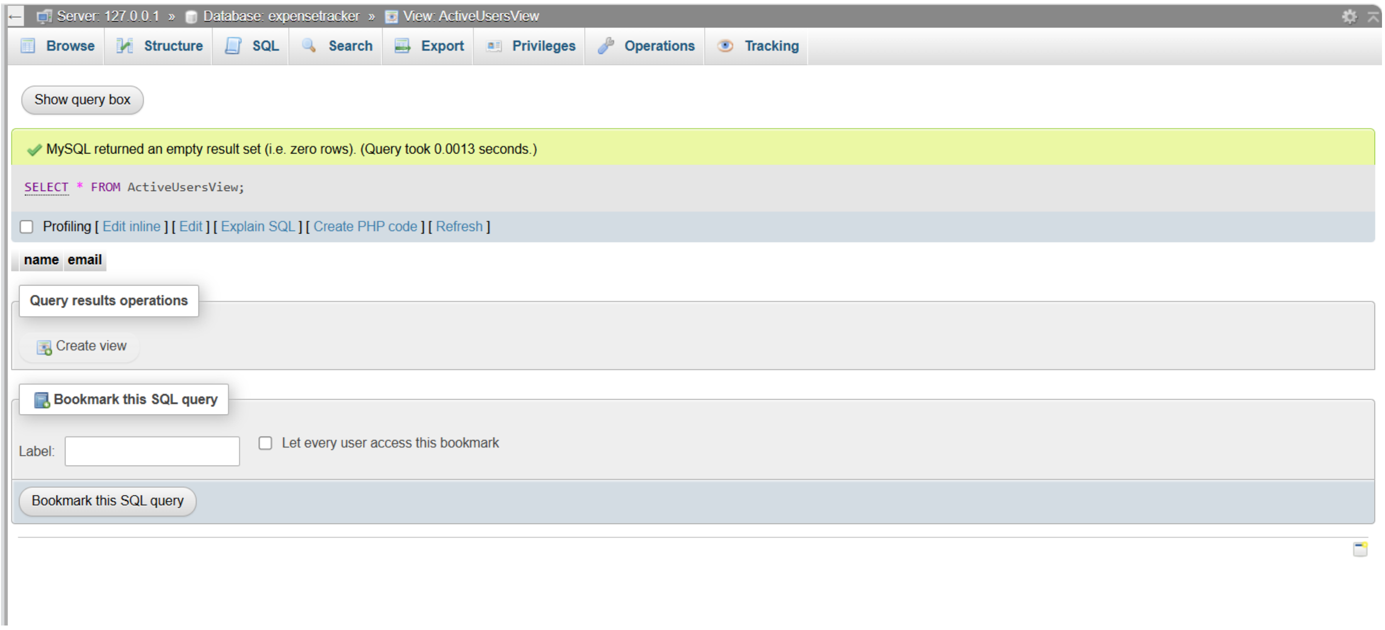
- **Display users sorted by total spending: List user names and their total sum of expenses, ordered from highest to lowest.**



4. Views

- **Create a view named ActiveUsersView : This view should list the name and email of any user who has logged more than 5 individual expense records.**

- **Query the view: Write a simple SELECT statement to show all data from ActiveUsersView.**

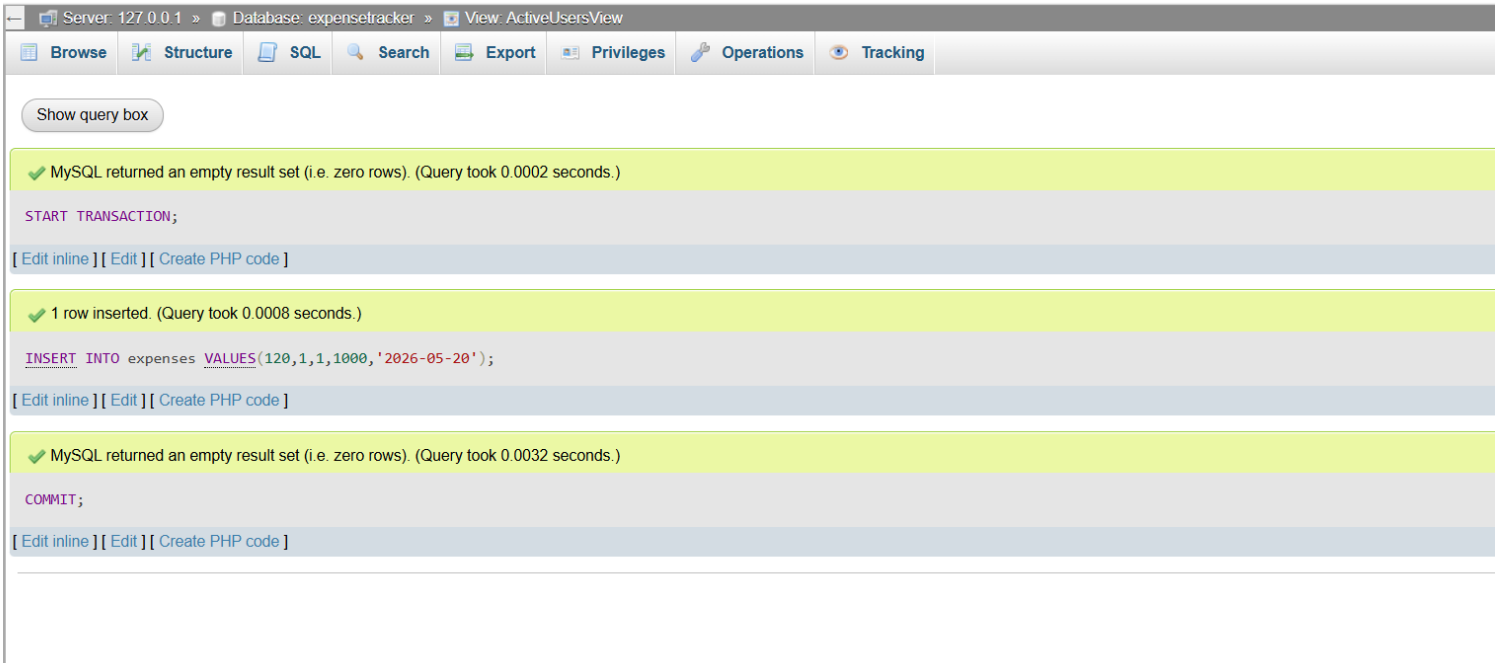


Section C: Mini Project

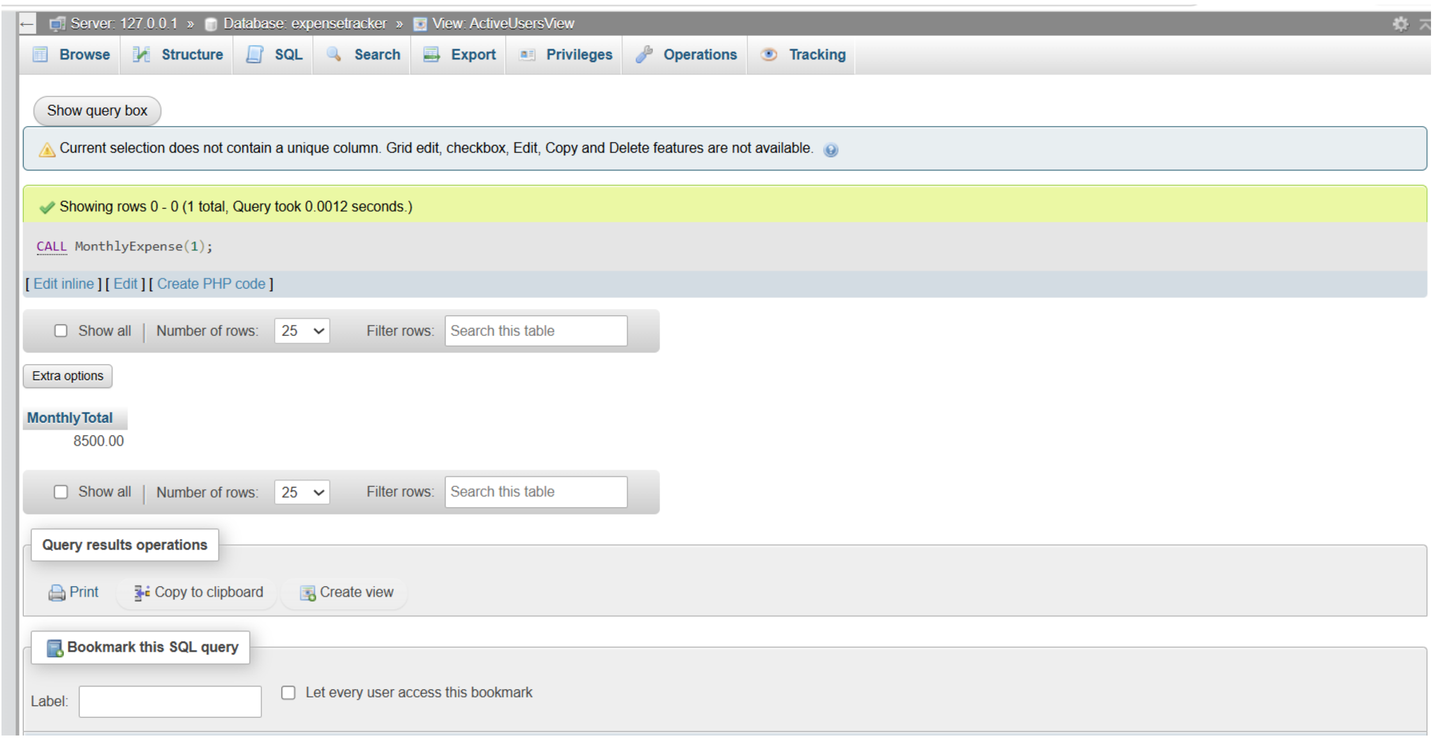
Mini Project: Expense Tracker DB

Using the same schema:

- **Write CRUD queries.**



- **Write a stored procedure to calculate monthly user expense**



- **Demonstrate COMMIT and ROLLBACK with example queries**